

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра вычислительных систем

Отчет по курсовой работе
по дисциплине
Архитектура распределенных приложений

по теме:
АВТОМАТИЗАЦИЯ ДЕПЛОЯ ПРИЛОЖЕНИЯ С ПОМОЩЬЮ HELM

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватель:
Преподаватель

Т.В Челканова

Новосибирск 2025 г.

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ	3
0.1 Цель	3
0.2 Задание.....	3
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	4
1.1 Постановка проблемы.....	4
1.2 Что такое Helm?.....	4
1.3 Что такое Chart?.....	4
1.4 Что такое Helm + Chart.....	5
2 ХОД РАБОТЫ	6
2.1 Доработка GITLAB CI-CD.....	6
2.2 Разработка deployment.yaml	8
2.3 Разработка service.yaml	9
2.4 Разработка ingress.yaml	10
2.5 Разработка Chart.yaml	10
2.6 Разработка values.yaml	10
3 ВЫВОД	12

ЦЕЛЬ И ЗАДАЧИ

0.1 Цель

Познакомиться с таким инструментом как helm и с помощью него организовать автоматический деплой приложения на сервер.

0.2 Задание

:

1. Познакомиться с helm.
2. Доработать CI/CD таким образом, чтобы осуществлялся автоматический деплой приложения на сервер.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Постановка проблемы

В лабораторной работе №3 был реализован деплой на сервер только одной версии приложения. Но что делать, если мы хотим задеплоить новую версию приложения? В этом случае придется менять манифест `deployment.yaml`, применять его с помощью `kubectl apply`. А если мы хотим сделать новый instance? Тогда придется переписывать `deployment.yaml`, `service.yaml`, `ingress.yaml`, а потом каждый применять через `kubectl`. Все это неудобно и долго. Хотелось бы автоматизировать этот процесс и упростить. Эту задачу решает `helm`.

1.2 Что такое Helm?

Менеджер пакетов для Kubernetes, который позволяет описывать приложение в виде `chart`'а и управлять его установкой, обновлением и версионированием.

Задачи Helm:

1. Устанавливать приложение в Kubernetes одной командой.
2. Обновлять приложение.
3. Удалять приложение.
4. Хранить историю версий приложения.
5. Откатываться назад.

1.3 Что такое Chart?

Helm chart описывает структуру и параметры Kubernetes-приложения и позволяет единообразно, повторяемо и управляемо разворачивать его в разных окружениях и в нескольких экземплярах.

1.4 Что такое Helm + Chart

Вместе они решают поставленную проблему: позволяют шаблонизировать манифесты кубера и во время деплоя в этот шаблон подставлять актуальные значения (а не хардкодить их, как в лабе №3). Помимо этого helm автоматически применяет манифесты и деплоит приложение.

ХОД РАБОТЫ

2.1 Доработка GITLAB CI-CD

Добавили этапы diff и deploy. Этап diff с помощью helm template показывает изменения, которые возникнут в манифестах при деплое. Этап deploy реализует сам deploy с помощью helm upgrade --install.

```
stages:
  - build
  - diff
  - deploy

build:
  stage: build
  image: docker:24
  services:
    - name: docker:24-dind
  before_script:
    - echo "$HAWK_REGISTRY_PASSWORD" | docker login -u
      "$HAWK_REGISTRY_USER" --password-stdin "$HAWK_REGISTRY"
    - docker context create context
    - docker buildx create --name mybuilder --driver
      docker-container --use context
  script:
    - TAGS="$HAWK_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
    - if [ -n "$CI_COMMIT_TAG" ]; then TAGS="$TAGS -t $
      HAWK_REGISTRY_IMAGE:$CI_COMMIT_TAG"; fi
    - docker buildx build --platform linux/amd64,linux/arm64 -f
      ./Dockerfile ./ -t $TAGS --push
    - echo "TAGS=${TAGS}" > tags.env
  rules:
    - if: "$CI_COMMIT_TAG"
      when: always
    - when: manual

artifacts:
  reports:
    dotenv: tags.env
```

```

diff:
  stage: diff
  image:
    name: alpine/helm:3.14.0
    entrypoint: [""]

  variables:
    NAMESPACE: "ia331"

  before_script:
    - apk add --no-cache curl
    - curl -LO "https://dl.k8s.io/release/$(curl -L -s
      https://dl.k8s.io/release/stable.txt)/bin/linux/arm64/kubectl"
    - chmod +x kubectl
    - mv kubectl /usr/local/bin/kubectl

  script:
    - mkdir -p ~/.kube
    - echo "$KUBECONFIG" > ~/.kube/config

    - helm template ia331s08-app ./chart --namespace
      "$NAMESPACE" --set image="$TAGS" | kubectl diff -n
      "$NAMESPACE" -f - || true

deploy:
  stage: deploy
  image:
    name: alpine/helm:3.12.3
    entrypoint: [""]

  variables:
    RELEASE_NAME: "ia331s08-app"

  script:
    - mkdir -p ~/.kube
    - echo "$KUBECONFIG" > ~/.kube/config
    - helm upgrade --install "$RELEASE_NAME" ./chart --namespace
      ia331 --set image="$TAGS"

  rules:
    - when: manual

```

2.2 Разработка deployment.yaml

Манифесты нужно превратить в шаблоны, значения в которые будут подставляться из файла values.yaml. Для шаблонизации заменим хардкодные значения на, например, `.Values.owner`. Это означает, что из файла values.yaml нужно взять переменную owner и подставить в это место. Таким образом манифесты становятся шаблонами.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
  namespace: {{ .Values.namespace }}
  labels:
    owner: {{ .Values.owner }}
spec:
  replicas: {{ .Values.replicas }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
        owner: {{ .Values.owner }}
    spec:
      serviceAccountName: ci
      containers:
        - name: {{ .Release.Name }}
          image: {{ .Values.image }}
          ports:
            - containerPort: 8000
          env:
            - name: PORT
              value: "8000"
          securityContext:
            allowPrivilegeEscalation: false
      resources:
        requests:
```



```

        memory: "32Mi"
        cpu: "32m"
    limits:
        memory: "64Mi"
        cpu: "64m"
    readinessProbe:
        httpGet:
            path: /
            port: 8000
        initialDelaySeconds: 5
        periodSeconds: 10
    livenessProbe:
        httpGet:
            path: /
            port: 8000
        initialDelaySeconds: 15
        periodSeconds: 20

```

2.3 Разработка service.yaml

Для более гибкой шаблонизации можно использовать конструкции вида `printf "%s-svc".Release.Name`. Эта конструкция позволяет вместо `%s` подставить `.Release.Name`. Это нужно для формирования имени service (и не только) на основе `.Release.Name`, что позволяет создавать instance без внесения изменений в `values.yaml`

```

apiVersion: v1
kind: Service
metadata:
  name: {{ printf "%s-svc" .Release.Name }}
  namespace: {{ .Values.namespace }}
  labels:
    owner: {{ .Values.owner }}
spec:
  selector:
    app: {{ .Release.Name }}
  ports:
    - port: 80
      targetPort: 8000

```

2.4 Разработка ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ printf "%s-ingress" .Release.Name }}
  namespace: {{ .Values.namespace }}
  labels:
    owner: {{ .Values.owner }}
spec:
  ingressClassName: {{ .Values.ingress.className }}
  rules:
    - host: {{ printf "%s.%s" .Release.Name
      .Values.ingress.domain }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ printf "%s-svc" .Release.Name }}
                port:
                  number: 80
```

2.5 Разработка Chart.yaml

```
apiVersion: v2
name: ia331s08-app
description: ia331s08-app
type: application
version: 0.1.0
appVersion: "1.0.0"
```

2.6 Разработка values.yaml

```
owner: ia331s08
namespace: ia331

deployment:
```

```
    replicas: 2

ingress:
  className: traefik-dev
  domain: hawk-dev.csc.sibsutis.ru
```

ВЫВОД

В ходе работы я познакомился с Helm и Chart - вспомогательными инструментами при работе с Kubernetes. С помощью изученных инструментов реализовал автоматизированный процесс деплоя приложения на кластер.